

Software Design Report
CSC 221: Programming 1: Spring 2024
Abraham Guernsey, Andre Goncalves, Tiago Silva
Lab 5: Inheritance Lab: Tic Tac Toe, Othello, Breakthrough, and Connect 4

Problem Summary

Testing games of Tic Tac Toe, Othello, Breakthrough, and Connect 4 each allowing for two people to play

Implementation Requirement

- Unit testing for each function required
- Integration testing by using the *GameDriver.java* program
- Condition coverage required whenever there is a boolean returned
- Statement coverage for all lines in each class
- Allow for two players to play
- Played in text mode
- Use GitHub to share code
- Must share super classes
- Use code from previous labs where applicable

System Design



Acceptance Testing Plan

	Class to Test	Name of Test	Description	Code for Test	Expected Result	Type of win (TTT)
Unit Testing	Player	testConst	Test the constructor given String and char	player1 = new Player("Joe Bob", 'X')		
				player1.toString()	Name: John, Symbol: X	
		testAcc	Test the accessor methods	player1 = new Player("Joe Bob", 'X')		
				player1.getName()	John	
				player1.getSymbol()	X	
		testEq2	Test all equal cases with a Player	player1 = new Player("John", 'X')		
				player2 = new Player("John", 'X')		
				player3 = new Player("James", 'X')		
				player4 = new Player("John", 'O')		
				player5 = new Player("James", 'O')		
				player1.equals(player2)	TRUE	
				player1.equals(player3)	FALSE	
				player1.equals(player4)	FALSE	
				player1.equals(player5)	FALSE	
		testConstErr	Test not giving correct values for constructor	player1 = new Player("John", 1)	Wrong value given for parm 2	
		testConstErr2	Test giving too many values for constructor	player1 = new Player("John", 'X', 1)	Too many values given	

					[[[[[[[[[[	
		testIsLegal	Test the isLegal method	othelloBoard = new OthelloBoard()		
				move = new Move(1,1)		
				othelloBoard.isLegal(move)	True (true if the move is legal and false if it is not)	
		testMakeMove	Test the makeMove method	othelloBoard = new OthelloBoard()		
				move = new Move(1,1)		
				player = new Player("a",'X')		
				othelloBoard.makeMove(move,player)	The Board makes the desired move	
		testGameOver	Test the gameOver method	othelloBoard = new OthelloBoard()		
				othelloBoard.gameOver()	False (return true if the game is over and false if it's not)	
		testIsWinner	Test the isWinner method	othelloBoard = new OthelloBoard()		
				othelloBoard.isWinner('X')	False (return true if the player associated with the desired piece wins the game and false if it did not)	
		testReplacePiece	Test the replacePiece method	othelloBoard = new OthelloBoard()		
				player1 = new Player("a",'X')		

				player2 = new Player("b",'O')		
				piece1 = new Piece(player1,'X')		
				piece2 = new Piece(player2,'O')		
				othelloBoard.replacePiece(piece 1,piece2	The program replaces the piece 1 for the piece 2	
		testIsTie	Test the isTie method	othelloBoard = new OthelloBoard()		
				othelloBoard.isTie()	True (return true if the game ends in a tie and false if it does not)	
	OthelloGame	testConstructor	Test the constructor with player names and type	Game othelloGame = new Game("A", "B", 2);	Create game	
		testConstFile	Test the constructor with filename	Game othelloGame = new Game("example.txt");	Game file	
		testGetCurrent	Test the getCurrent() method	Game othelloGame = new Game("A", "B", 2); currentPlayer = othelloGame.getCurrent();	"A" or "B" depending on the scenario	
		testPlay	Test the play() method	Game othelloGame = new Game("A", "B", 2); othelloGame.play();	Depending on game outcome, either player wins or draws	
		testEndGame	Test the endGame() method	Game othelloGame = new Game("A", "B", 2); othelloGame.endGame()	Game ends without errors	

		testToString	Test the toString() method	Game othelloGame = new Game("A", "B", 2); othelloGame.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>	
	OthelloMove	testConst	Test Constructor	Move othelloMove1 = new Move(0,6)		
		testGetColumn	Test getColumn() method	column = move.getColumn()	0	
		testGetRow	Test getRow() method	column = move.getRow()	6	
		testEquals	Test equals() method	Move othelloMove1 = new Move(0,6)		
				Move othelloMove2 = new Move(0,6)		
				Move othelloMove3 = new Move(0,5)		
				othelloMove1.equals(othelloMove2)	TRUE	
				othelloMove1.equals(othelloMove3)	FALSE	
		testToString	Test toString() method	OthelloMove1.toString()	Row:6 Column:0	
		testConst1	Test Constructor given nothing	board = new Connect4Board()		
				board.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>	

Connect4Board

		testAcc	Test the accessor methods	board = new Connect4Board()		
				move = new Move(1)		
				board.getPiece(move)	R	
		testIsLegal	Valid Move	board = new Connect4Board()		
				move = new Move(1)		
				player1 = new Player("Tiago", 'R')		
				board.makeMove(move, player1)		
				board.toString()	<pre> _ R _ _ _ </pre>	
		testIsLegal	Invalid move	board = new Connect4Board()		
				move = new Move(7)		
				player1 = new Player("Tiago", 'R')		
				board.makeMove(move, player1)	Out of bounds.	
		testMakeMove	Test move	board = new Board()		
				move = new Move(1)		
				player1 = new Player("Tiago", 'R')		

				board.makeMove(move, player1)		
				board.toString()	<pre> _ R _ _ _ </pre>	
		testGameOver	Game is not over	board = new Connect4Board()		
				player1 = new Player("Tiago", 'R')		
				board.toString()	<pre> _ R _ _ _ </pre>	
					Game is not over yet	
			Game is over	board = new Connect4Board()		
				player1 = new Player("Tiago", 'R')		
				player2 = new Player("Ana", 'Y')		
				board.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ R Y _ _ R Y Y _ Y R Y Y R _ </pre>	
					Game over!	
		testIsWinner	Winner	board = new Connect4Board()		
				player1 = new Player("Tiago", 'R')		

				player2 = new Player("Ana", 'Y')		
				board.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ _ R Y _ _ _ R Y Y _ Y R Y Y R _ </pre>	
					Tiago wins!	
		testIsTie	The game tied	board = new Connect4Board()		
				player1 = new Player("Tiago", 'R')		
				player2 = new Player("Ana", 'Y')		
				board.toString()	<pre> Y R R Y R Y R Y R Y R Y Y R Y R Y R Y R Y R Y Y R Y R Y Y R Y R Y Y R Y </pre>	
					It's a tie!	
	Connect4Move	testConst	Test Constructor	Move move1 = new Move(3)		
		testGetColumn	Test getColumn() method	column = move.getColumn()	3	
		testEquals	Test equals() method	Move move2 = new Move(3)		
				move1.equals(move2)	TRUE	
		testToString	Test toString() method	move.toString()	Column: 3	

	Connect4Game	testConstructor	Test the constructor with player names and type	Game game = new Game("Tiago", "Ana", 1);	create game	
		testConstFile	Test the constructor with filename	Game game = new Game("example.txt");	Game file	
		testGetCurrent	Test the getCurrent() method	currentPlayer = game.getCurrent();	"Tiago" or "Ana" depending on the scenario	
		testPlay	Test the play() method	game.play();	Depending on game outcome, either player wins or draws	
		testTakeTurn	Test the takeTurn() method with valid move	game.takeTurn(Tiago, R, 2);	Tiago's turn	
		testEndGame	Test the endGame() method	game.endGame()	Game ends without errors	
		testToString	Test the toString() method	game.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>	
	TicTacToeBoard	testConst1	Test Constructor given nothing	board = new TTTBoard()		
				board.toString()	<pre> _ _ _ _ </pre>	
		testConst2	Test the constructor given an int	board = new TTTBoard(3)		
				board.toString()	<pre> _ _ _ _ </pre>	

		testConst3	Test constructor with board bigger than 3	board1 = new TTTBoard(10)		
				board1.toString()	10x10 Board	
		testAcc	Test the accessor methods	board = new TTTBoard(3)		
				move = new Move(1,1)		
				board.getPiece(move)	X	
		testAdd	Test adding a piece	board = new TTTboard(3)		
				move = new Move(1,1)		
				player1 = new Player("John", 'X')		
				board.makeMove(move, player1)		
				board.toString()	X _ _ _ _ _ 	
	TicTacToeGame	testWinCases	Testing all different test cases	X O _ X O _ X	X wins	(Vertical 1)
				X O _ X O X O	O wins	(Vertical 2)
				O _ X O _ X X	X wins	(Vertical 3)
				O O O X X _ X	O wins	(Horizontal 1)

				O O X X X 	X wins	(Horizontal 2)
				X _ X X _ O O O	O wins	(Horizontal 3)
				X O _ O X _ X	X wins	(Diagonal 1)
				X O X O X O	O wins	(Diagonal 2)
				X O O O X X X X O	Tie	(Tie)
		toString	To String of a game	System.out.println(game.toString ());	3x3 board, Name: a, Symbol: X, Name: b, Symbol: O	
		testFile	Test using a file as input	input = new IODriver()		
				player1 = new Player(input.getString("Getting X Player name."), 'X')		
				player2 = new Player(input.getString("Getting O Player name."), 'O')		
				board = new TTTBoard(input.getInt("Getting the size"))		
				player1.toString()	Name: Joe Bob, Symbol: X	
				player2.toString()	Name: Bob Joe, Symbol: O	
				board.toString()	_ _ _ _ 	
		testMoveErr	Test moving to an	With a 3x3 board, adding a piece at 1,1 then trying to add the next piece to 1,1 as well	Asks for row and column again until correctly	

			occupied space		inputted	
		testMoveErr2	Test moving off the board	With a 3x3 board, adding a piece at 4,4	Asks for row and column again until correctly inputted	
	Breakthrou ghBoard	testConst1	Test Constructor given nothing	board = new BreakthroughBoard()		
				board.toString()	<pre> o o o o o o o o o o o o o o o o _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ x x x x x x x x x x x x x x x x </pre>	
		testConst2	Test constructor with a different size	board = new BreakthroughBoard(9)		
				board.toString()	<pre> o o o o o o o o o o o o o o o o o o _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ x x x x x x x x x x x x x x x x x x </pre>	
		testIsLegal	Valid Move	board = new BreakthroughBoard()		
		(also makeMove)		move = new Move(0, 6, 0, 5)		
				player1 = new Player("Abe", 'x')		
				board.makeMove(move, player1)		
				board.toString()	<pre> o o o o o o o o o o o o o o o o _ _ _ _ _ _ _ _ </pre>	

					<pre> _ _ _ _ _ _ _ _ _ _ x _ _ _ _ _ _ x x x x x x x x x x x x x </pre>	
		testIsLegal	Invalid move	board = new BreakthroughBoard()		
				move = new Move(8, 8, 8, 8)		
				player1 = new Player("Abe", 'x')		
				board.makeMove(move, player1)	Out of bounds.	
		testGameOver	Game is not over	board = new BreakthroughBoard()		
				move = new Move(0, 6, 0, 5)		
				board.toString()	<pre> o o o o o o o o o o o o o o o o _ x x x x x x x x x x x x x x </pre>	
				gameOver()	Game is not over yet	
			Game is over	board = new BreakthroughBoard()		
				(Play game -> end)		
				board.toString()	<pre> o o o o o x o o o o o _ _ o _ x x x x x _ x x x x x x x x x </pre>	

				gameOver()	Game over!	
		testIsWinner	Winner	board = new BreakthroughBoard()		
				(Play game -> end)		
				board.toString()	<pre> o o o o o x o o o o o o _ o _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ x x x x x _ x x x x x x x x </pre>	
					x wins	
		testIsTie	The game tied	isTie()	null (tie is not possible in this game)	
	Breakthrough Move	testConst	Test Constructor	Move move1 = new Move(0,6,0,5)		
		testGetColumn n	Test getColumn() method	column = move.getColumn()	0	
		testGetColumn n2	Test getColumn 2() method	column = move.getColumn2()	0	
		testGetRow	Test getRow() method	column = move.getRow()	6	
		testGetRow2	Test getRow2() method	column = move.getRow2()	5	
		testEquals	Test equals() method	Move move2 = new Move(0,6,0,5)		
				Move move3 = new Move(0,5,0,5)		

		Should also test each row, row2, column, and column2 being different and the same for conditional testing, but this would take				
				move1.equals(move3)	FALSE	
				move1.equals(move2)	TRUE	
		testToString	Test toString() method	move1.toString()	From: row: 6, column: 0 To: row: 5, column: 0	
	BreakthroughGame	testConstructor	Test the constructor with player names and type	Game game = new Game("Abe", "B", 3);	create game	
		testConstFile	Test the constructor with filename	Game game = new Game("example.txt");	Game file	
		testGetCurrent	Test the getCurrent() method	currentPlayer = game.getCurrent();	"Abe" or "B" depending on the scenario	
		testPlay	Test the play() method	game.play();	Depending on game outcome, either player wins or draws	
		testEndGame	Test the endGame() method	game.endGame()	Game ends without errors	
		testToString	Test the toString() method	game.toString()	<pre> o o o o o o o o o o o o o o _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>	

					<pre> _ _ _ _ _ _ _ _ _ _ x x x x x x x x x x x x x x </pre>	
		testWinCases	Test the different ways to win	An x piece gets to the other side	x wins	
				An o piece gets to the other side	o wins	
		testTaking	Test pieces taking another	<pre> o o o o o o o o o o o _o o o _ x x x x x x x x x x x x x x </pre>	x piece below o on the 6th row cannot take the o, but the x pieces below and to the left and right are able to	

Time Spent Estimate

~4 hours expected, each project might differ slightly based on difficulty of the game

Potential Outside Resources used

- Code from previous labs
- Code from Dr. Bareiss' solutions
- Members
 - Tiago - Connect 4: problem summary, helped with UML, and improvements on work, testing for game
 - Andre - Othello: UML, time spent, ethical report, testing for game
 - Abraham - Breakthrough: UML, implementation requirement, security risks, lessons learned, testing for game and tic tac toe

Potential Security Risks

- There will be accessors for most classes, although they should be avoided
- People can use the file input to find if a file is in a system
- Some of the classes will not be final, so subclasses can be made of them

Potential Ethical Reports

- **Ethical use:** Can be used to let people play fun games of Tic Tac Toe, Othello (Reversi), Connect 4, and Breakthrough
- **Unethical use:** No unethical use known

Lessons Learned

- How to reason through if classes should be super/sub classes or if they need to be abstract or interfaces

Improvements on work

- Fix small mistakes on the testing plan.